European Journal of Operational Research 62 (1992) 203–209
North-Holland

Theory and Methodology

A multicriteria Pareto-optimal path algorithm

Chi Tung Tung and Kim Lin Chew
Department of Mathematics, National University of Singapore, Singapore

Received June 1990; revised July 1990

Abstract: In this paper we present an algorithm for finding, in a multicriteria network, Pareto-optimal paths, one each for each efficient objective vector. Validation of the algorithm and two illustrative examples are included. Computational experience is also reported. This algorithm is a generalization of an earlier one for the bicriterion case.

Keywords: Network; algorithm; multicriteria Pareto-optimal path; heuristics

1. Introduction

In a network optimization problem, it is often found that a single objective function is not sufficient to characterize adequately the problem. Bicriterion or more generally multicriteria setting seems to be more suitable for many problems dealing with the design and efficient use of transportation, transmission and communication networks.

Let $[N; A]$ be a connected network, where N is the set of nodes and A the set of indirected and directed arcs. Associated with each arc $(x, y) \in A$, there is a cost vector $c(x, y)$ of m positive components $c_1(x, y), c_2(x, y), \ldots, c_m(x, y)$, giving rise to m objective functions $c_1, c_2, \ldots, c_m$. These components may represent cost, time, energy consumed, degree of risk and so on in practical problems. A path P_{uv} from node u to node v is a sequence

$$P_{uv}: u = x_1, x_2, \ldots, x_k = v \tag{1}$$

of nonrepeated nodes $x_1, x_2, \ldots, x_k$; and connecting arcs $(x_1, x_2), (x_2, x_3), \ldots, (x_{k-1}, x_k)$ connecting u to v. In this paper, we are only concerned with loopless paths [7]. We assume that our network is strongly connected in the sense that for any $u, v \in N$, there exists at least one path which connects u to v.

For a path as given in (1), we associate an objective vector $c(P_{uv})$ whose j-th component is given by

$$c_j(P_{uv}) = \sum_{i=2}^{k} c_j(x_{i-1}, x_i). \tag{2}$$

A path P_{uv} from node u to node v *dominates* another path Q_{uv} if the objective vector $c(P_{uv})$ dominates $c(Q_{uv})$ in the following sense:

$$c_j(P_{uv}) \leqslant c_j(Q_{uv}), \quad j = 1, 2, \ldots, m \tag{3}$$

where at least one j makes the inequality true.

The path P_{uv} is a *Pareto-optimal* path (i.e. an efficient path) from node u to node v and its associated objective vector is *efficient* if there exists no other path Q_{uv} from u to v which dominates P_{uv}. The *multicriteria shortest path problem* in this paper is to find the set Γ of all

0377-2217/92/$05.00 © 1992 – Elsevier Science Publishers B.V. All rights reserved

efficient objective vectors, and to find for each efficient objective vector a Pareto-optimal path from a prescribed source node s to a prescribed terminal node t in a network $[N; A]$.

The single-criterion shortest path problem has become classic. For the bicriterion shortest path problem, the algorithms of Hansen [4] and Vincke [12] typically make use of a multiple labelling scheme. Corley and Moon [2] gave a modification of Ford's algorithm [6] to find all Pareto-optimal paths from a specified node to every other node. Climaco and Martins [1,5] used a K-shortest paths routing to solve the problem. Tung and Chew [11] have recently applied a multiple labelling scheme coupled with a heuristic evaluation concept [9,10] to solve the problem. The evaluation function used for search guidance is expressed in terms of the sum of the objective functions. In fact, any strictly increasing function of the objective functions can be used instead.

For the multicriteria shortest path problem ($m > 2$), Martins [8] gave a generalization of the multiple labelling scheme of Hansen for bicriteria cases. The method used by him is basically lexicographically ordered search. In this paper, we present a generalization of the algorithm which has been employed for the bicriterion case [11].

2. The algorithm

Consider a strongly connected network $[N; A]$, where each arc (x, y) is associated with a cost vector $c(x, y) = (c_1(x, y), c_2(x, y), \ldots, c_m(x, y))$ and each path P_{uv} is associated with an objective vector $c(P_{uv}) = (c_1(P_{uv}), c_2(P_{uv}), \ldots, c_m(P_{uv}))$, $c_j(P_{uv})$ being given by (2). To find efficient paths from a prescribed node s to another prescribed node t, we proceed by building up step by step, starting from s, loopless paths through alternating branching and evaluating processes. As we progress, each node z will be given in succession a sequence of labels: $\theta^1(z)$, $\theta^2(z), \ldots$. Each time a node is reached, it is given the label $\theta^k(z)$ which consists of two parts:

$$\theta^k(z) = [x^i, \alpha^k(z)], \tag{4}$$

indicating that node z is reached from node x; the vector $\alpha^k(z)$ is obtained from the i-th label of node x using the relation

$$\alpha^k(z) = \alpha^i(x) + c(x, z) \tag{5}$$

where $\alpha^i(x)$ is the objective vector associated with the i-th label of node x which has just been extended to z. For ease of reference, we call $\theta^k(z)$ the *path tip* of the path just constructed at z. The size of the set of all labels given to a node is bounded by

$$\sum_{k=0}^{n-2} \binom{n-2}{k} k! \tag{6}$$

where n is the size of N.

To evaluate a node with a particular label, we propose to use the sum of the objective functions as a guide. In fact, any strictly increasing function of the objective functions can also be used.

Given a vector $l = (l_1, l_2, \ldots, l_m)$, we denote the sum of the components of l by tr l:

$$\text{tr } l = l_1 + l_2 + \cdots + l_m. \tag{7}$$

Given an arc (x, y), we define $h(x, y)$ by setting

$$h(x, y) = \text{tr } c(x, y). \tag{8}$$

For a path P_{uv} connecting u and v, we define $h(P_{uv})$ by setting

$$h(P_{uv}) = \text{tr } c(P_{uv}) \tag{9}$$

where $c(P_{uv})$ is the objective vector given in (2). We also define

$$q_j(u, v) = \min c_j(P_{uv}), \tag{10}$$

$$h^*(u, v) = \min h(P_{uv}) \tag{11}$$

where each minimum is taken over all paths from u to v. Let $q(u, v)$ be the vector with components $q_j(u, v)$. For a label $\theta^k(z)$ at node z as given in (4), we set the evaluation value $e^k(z)$ as

$$e^k(z) = \text{tr } \alpha^k(z) + h^*(z, t). \tag{12}$$

Thus, $e^k(z)$ is the sum of the cost components of the path from s to z already constructed plus the minimum of the sum of the cost components among all possible paths from z to t.

A label when first obtained is a *tentative* label. The set of all tentative labels for node z is

denoted by $L(z)$. Let L be the union of all $L(z)$ over all nodes z. A tentative label becomes permanent when it is chosen in the evaluation process.

We shall describe below an algorithm for finding the set Γ of all efficient objective vectors, and a Pareto-optimal path for each efficient objective vector.

Before we start the algorithm, we may compute $q_j(z, t)$ $(j = 1, 2, \ldots, m)$ and $h^*(z, t)$. This can be achieved by reversing directions of all arcs and applying Dijkstra's algorithm [3] to find the shortest routes from t to other nodes in the resulting network. The reason is as follows. Let $[N; A]'$ denote the network with all directed arcs reversed in the network $[N; A]$. Given any path from a node z to the terminal node t in a network $[N; A]$, there is only one corresponding path from node t to node z in the network $[N; A]'$, simply by back-tracing the one in the network $[N; A]$. Therefore, the 'shortest path' from a node z to the terminal node t in the network $[N; A]$ is the same path (but reverse direction) as the shortest path from t to z in the network $[N; A]'$. Hence, for saving computing time we apply Dijkstra's algorithm on the network $[N; A]'$ with considering t as the starting node to find (for all $z \in N$) $q_1(z, t)$, $q_2(z, t), \ldots,$ $q_m(z, t)$ and $h^*(z, t)$ one by one.

To initialize the algorithm of finding efficient paths from node s to t, we set $\theta^1(s) = [-; \alpha^1(s)]$, where $\alpha^1(s) = 0$, $e^1(s) = h^*(s, t)$, $L(s) = \{\theta^1(s)\}$, $L(y) = \emptyset$ for $y \neq s$, and $\Gamma = \emptyset$.

At an iterative step, consider the set L of all current tentative labels. If L is empty, stop the execution. Otherwise, choose a tentative label $\theta^\nu(x)$ having the minimal evaluation value among all the tentative labels. Let the tentative label $\theta^\nu(x)$ become permanent and remove it from $L(x)$.

Case 1. If $x = t$, determine if $\alpha^\nu(t)$ is an efficient objective vector by directly testing with the efficient objective vectors already found. If it is, include $\alpha^\nu(t)$ in Γ. Trace back from the path tip $\theta^\nu(t)$ to obtain the Pareto-optimal path associated with $\alpha^\nu(t)$.

The vector $\alpha^k(t)$ tested is in fact an efficient objective vector if it is accepted by all the efficient objective vectors already found. This will be justified in the next section.

Case 2. If x is not t, consider the neighboring nodes of x. Reject a neighboring node z if it is already on the path with the path tip $\theta^\nu(x)$, or the following conditions is satisfied:

$$\alpha^\nu(x) + c(x, z) + q(z, t) \geqslant \xi \tag{13}$$

where ξ is some efficient objective vector already found. For each node z not rejected, assign a tentative label as follows: if $k - 1$ (tentative and permanent) labels have been given to z on previous occasions, assign the label $\theta^k(z) = [x^\nu; \alpha^k(z)]$ to z, where $\alpha^k(z) = \alpha^\nu(x) + c(x, z)$; set $L(z) := L(z) \cup \{\theta^k(z)\}$, and compute the evaluation value $e^k(z)$ given in (12). Repeat the iterative step until L is empty.

We summarize the algorithm in the following steps:

Step 1. Apply Dijkstra's algorithm on the network with all arcs reversed to find $q_i(z, t)$ $(i = 1, 2, \ldots, m)$ and $h^*(z, t)$ for all $z \in N$.

Step 2. Assign to s the tentative label $\theta^1(s) = [-; \alpha^1(s)]$ where $\alpha^1(s) = 0$. Set $e^1(s) = h^*(s, t)$, $L(s) = \{\theta^1(s)\}$, $L(y) = \emptyset$ for $y \neq s$, and $\Gamma = \emptyset$.

Step 3. Set $L = \cup_y L(y)$. If $L = \emptyset$, stop the execution. Otherwise, identify the tentative label $\theta^\nu(x) \in L$ having the minimal evaluation value $e^\nu(x)$ among all the tentative labels in L. (Choose arbitrarily if there is a tie.) Let $\theta^\nu(x)$ become permanent and set $L(x) := L(x) - \{\theta^\nu(x)\}$. In case $x = t$, go to Step 6. Otherwise, go to Step 4.

Step 4. Consider all the neighboring nodes of x. Reject a neighboring node z if it is already on the path with path tip $\theta^\nu(x)$, or condition (13) is true. Go to Step 3 if all the neighboring nodes of x are rejected; otherwise, go to Step 5.

Step 5. For each node z not rejected in Step 4, assign a tentative label as follows: if $k - 1$ (tentative and permanent) labels have been given to z on previous occasions, assign the label $\theta^k(z) = [x^\nu; \alpha^k(z)]$ to z where $\alpha^k(z) = \alpha^\nu(x) + c(x, z)$; set $L(z) := L(z) \cup \{\theta^k(z)\}$, and compute the evaluation value $e^k(z)$ given in (12). Go to step 3.

Step 6. Determine whether the vector $\alpha^\nu(t)$ is an efficient objective vector by directly testing with the efficient objective vectors in Γ. If it is, include $\alpha^k(t)$ in Γ. Trace back from the path tip $\theta^\nu(t)$ to find the associated Pareto-optimal path. Go to Step 3.

3. Justification of the algorithm

Starting from the prescribed source node s, the algorithm builds up loopless paths step by step ending at some path tips at the end of an iteration. At the beginning of the next iteration, it chooses a path tip according to the evaluation value and extends the path a step further to next path tips. It makes sure that the paths remain loopless. The rejection rule (13) prevents us from wasting time in finding paths which are not Pareto-optimal, or paths which are alternatives to a Pareto-optimal path with the same efficient objective vector.

When a path tip (or label) is chosen to branch off, it becomes and remains permanent, and so we can talk about the relative age among the permanent labels. If we arrange the permanent labels in a sequence in descending order of their ages, we observe that their evaluation values form an increasing sequence.

When the prescribed terminal node t is reached and given the label $\theta^{\nu}(t) = [x^k, \alpha^{\nu}(t)]$, which is the path tip of some loopless path, the objective vector $\alpha^{\nu}(t)$ cannot dominate any objective vectors already in the set Γ, for if $\alpha^{\nu}(t)$ dominates $\alpha^{\mu}(t)$ in Γ, the evaluation value

$$e^{\mu}(t) = \text{tr}\ \alpha^{\mu}(t) > \text{tr}\ \alpha^{\nu}(t) = e^{\nu}(t). \qquad (14)$$

But the permanent label $\theta^{\mu}(t)$ is older than $\theta^{\nu}(t)$, implying that $e^{\mu}(t) \leqslant e^{\nu}(t)$, which contradicts (14). Hence $\alpha^{\nu}(t)$ does not dominate any objective vectors in Γ. Since the entering objective vector does not dominate any existing objective vector in Γ, we conclude that Step 6 collects the set Γ exactly.

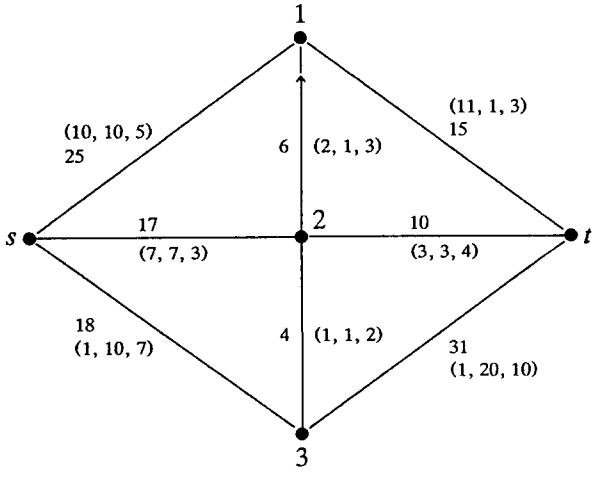


Figure 1. A network

Table 1

z	s	1	2	3	t
$q_1(z, t)$	2	11	2	1	0
$q_2(z, t)$	9	1	2	3	0
$q_3(z, t)$	7	3	4	6	0
$h^*(z, t)$	27	15	10	14	0

4. Two examples

Example 1 (see Figure 1)

In Step 1, the values of $q_i(z, t)$ and $h^*(z, t)$ for $i = 1, 2, 3$, and $z \in N$ are computed as shown in Table 1.

In Step 2, assign the tentative label $\theta^1(s) = [-; \alpha^1(s)]$ where $\alpha^1(s) = 0$. Set $e^1(s) = h(s, t)$, $L(s) = \{\theta^1(s)\}$, $L(y) = \emptyset$ for $y \neq s$, and $\Gamma = \emptyset$.

In Step 3, the current $L = \{\theta^1(s)\}$. The label $\theta^1(s)$ is chosen to become permanent.

In Step 4, consider the neighboring nodes 1, 2 and 3 to s. None of these are rejected.

In Step 5, assign to 1, 2 and 3 the following tentative labels, and compute the corresponding evaluation values:

$$\theta^1(1) = [s^1; (10, 10, 5)], \quad e^1(1) = 25 + 15 = 40,$$
$$\theta^1(2) = [s^1; (7, 7, 3)], \quad e^1(2) = 17 + 10 = 27,$$
$$\theta^1(3) = [s^1; (1, 10, 7)], \quad e^1(3) = 18 + 14 = 32.$$

Go to Step 3. The current $L = \{\theta^1(1), \theta^1(2), \theta^1(3)\}$. Since $\theta^1(2)$ has the smallest evaluation value, choose $\theta^1(2)$ to become permanent.

Go to Step 4. Consider the neighboring nodes s, 1, t and 3 to node 2. None of these are rejected except s.

Go to Step 5. Assign to 1, t and 3 the following tentative labels and compute the corresponding evaluation values:

$$\theta^2(1) = [2^1; (9, 8, 6)], \quad e^2(1) = 23 + 15 = 38$$
$$\theta^1(t) = [2^1; (10, 10, 7)], \quad e^1(t) = 27 + 0 = 27,$$
$$\theta^2(3) = [2^1; (8, 8, 5)], \quad e^2(3) = 21 + 14 = 35.$$

Go to Step 3. The current $L = \{\theta^1(1), \theta^1(3), \theta^2(1), \theta^1(t), \theta^2(3)\}$. The label $\theta^1(t)$ has the least evaluation value and is chosen to become permanent.

Go to Step 6. We have obtained an efficient vector, namely (10, 10, 7). The corresponding Pareto-optimal path is $s-2-t$. Repeating the steps, we obtain:

$L = \{\theta^1(1), \theta^1(3), \theta^2(1), \theta^2(3)\}$.

Let $\theta^1(3)$ become permanent. Compute

$$\theta^2(2) = [3^1; (2, 11, 9)], \quad e^2(2) = 22 + 10 = 32,$$
$$\theta^2(t) = [3^1; (2, 30, 17)], \quad e^2(t) = 49 + 0 = 49.$$

$L = \{\theta^1(1), \theta^2(1), \theta^2(3), \theta^2(2), \theta^2(t)\}$.

Let $\theta^2(2)$ become permanent.

The neighboring node 1 of 2 is rejected because rejection rule (13), (4, 12, 12) + (11, 1, 3) > (10, 10, 7). Nodes s and 3 are already on the path. There is only one node t which is not rejected. Compute

$$\theta^3(t) = [2^2; (5, 14, 13)], \quad e^3(t) = 32 + 0 = 32,$$

$L = \{\theta^1(1), \theta^2(1), \theta^2(3), \theta^2(t), \theta^3(t)\}$.

Let $\theta^3(t)$ become permanent. We obtain an efficient vector, namely (5, 14, 13). The corresponding Pareto-optimal path is $s-3-2-t$.

$L = \{\theta^1(1), \theta^2(1), \theta^2(3), \theta^2(t)\}$

Let $\theta^2(3)$ become permanent.

The neighboring nodes s and 2 of 3 are already on the path. Node t is rejected because by (13), (9, 28, 15) > (5, 14, 13).

$L = \{\theta^1(1), \theta^2(1), \theta^2(t)\}$.

Let $\theta^2(1)$ become permanent.

Node s is already on the path. Compute

$$\theta^4(t) = [1^2; (20, 9, 9)], \quad e^4(t) = 38 + 0 = 38.$$

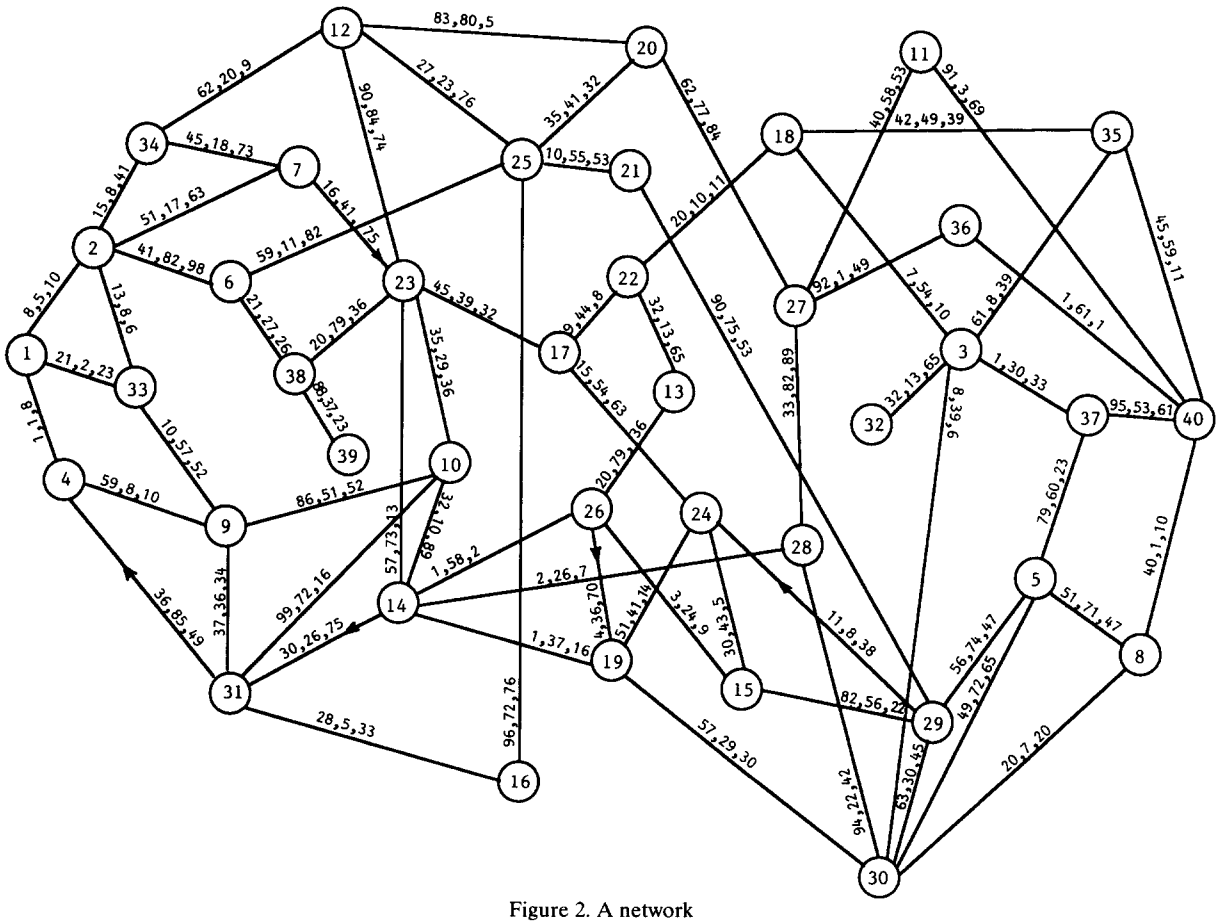


Figure 2. A network

$L = \{\theta^1(1), \theta^2(t), \theta^4(t)\}$.
Let $\theta^4(t)$ become permanent. We obtain an efficient vector, namely (20, 9, 9). The corresponding Pareto-optimal path is $s - 2 - 1 - t$.
$L = \{\theta^1(1), \theta^2(t)\}$
Let $\theta^1(1)$ become permanent.

The neighboring node t of 1 is rejected because by (13), (21, 11, 8) > (10, 10, 7).
$L = \{\theta^2(t)\}$.
Let $\theta^2(t)$ become permanent. We obtain an efficient vector, namely (2, 30, 17). Its corresponding Pareto-optimal path is $s - 3 - t$.
The set L is empty. Stop the execution.

We have obtained the set of all efficient objective vectors and the corresponding Pareto-optimal paths which are listed below:

(10, 10, 7)	(5, 14, 13)	(20, 9, 9)	(2, 30, 17)
$s-2-t$,	$s-3-2-t$,	$s-2-1-t$,	$s-3-t$.

Example 2

Given source node 2 and terminal node 37 of Figure 2, we have obtained the following result by using an IBM 3081 computer on the FORTRAN programme of our algorithm. The number given for P below is the number of path tips which have become permanent at that stage.
ROUTE 1 is:

$2 \to 7 \to 23 \to 17 \to 22 \to 18 \to 3 \to 37$.

Efficient objective vector = (149, 235, 232) and $P = 8$.
ROUTE 2 is:

$2 \to 1 \to 4 \to 9 \to 10 \to 14 \to 19 \to 30 \to 3 \to 37$.

Efficient objective vector = (253, 210, 254) and $P = 30$.
ROUTE 3 is:

$2 \to 1 \to 4 \to 9 \to 10 \to 14 \to 28 \to 30 \to 3 \to 37$.

Efficient objective vector = (291, 192, 257) and $P = 36$.
ROUTE 4 is:

$2 \to 1 \to 4 \to 9 \to 10 \to 23 \to 17 \to 22 \to 18 \to 3 \to 37$.

Efficient objective vector = (271, 271, 210) and $P = 42$.
ROUTE 5 is:

$2 \to 34 \to 12 \to 23 \to 17 \to 22 \to 18 \to 3 \to 37$.

Efficient objective vector = (249, 289, 218) and $P = 49$.
ROUTE 6 is:

$2 \to 7 \to 23 \to 10 \to 14 \to 19 \to 30 \to 3 \to 37$.

Efficient objective vector = (201, 232, 348) and $P = 69$.
ROUTE 7 is:

$2 \to 7 \to 23 \to 10 \to 14 \to 28 \to 30 \to 3 \to 37$.

Efficient objective vector = (239, 214, 351) and $P = 81$.
ROUTE 8 is:

$2 \to 1 \to 4 \to 9 \to 31 \to 10 \to 23 \to 17 \to 22 \to 18 \to 3 \to 37$.

Efficient objective vector = (321, 328, 208) and $P = 107$.
ROUTE 9 is:

$2 \to 1 \to 4 \to 9 \to 10 \to 14 \to 28 \to 30 \to 8 \to 40 \to 37$.

Efficient objective vector = (437, 184, 309) and $P = 124$.
L set is empty when $P = 132$.

5. Computational experience and conclusion

In this paper we present a new efficient algorithm for finding a complete set of all efficient paths (i.e. Pareto-optimal paths) from a preassigned source node s to a preassigned terminal node t in a network. Since one efficient objective vector may be correspond to more than one distinct efficient path, the complete set here is completed by collecting only one efficient path for every efficient objective vector.

Some of the discarding conditions used in the bicriterion case [11] are no longer applicable for the situation which has more than two objective functions (i.e. $m > 2$).

Although there are different units used in different objective functions we consider their sum in constructing the evaluation function $e^k(z)$ as stated in (12). The significances of this consideration are

i) There is a sense of average.
ii) Lexicographical order is not used.
iii) The method of A^* algorithm in heuristic search [10] is implemented.

iv) Discarding process is imposed whenever there is a chance at a path tip.

We have done some computations with our algorithm for a few examples (written in FORTRAN programme and executed by IBM 3081 computer). One of our examples is provided as Example 2 in this paper. For a network of the size such as Figure 2, the computational results (a complete set of efficient paths with its associated vectors) can be obtained on the computer terminal screen instantaneously right after entering the execution command.

Acknowledgement

The authors would like to thank the referee for his valuable and substantive suggestions.

References

[1] Climaco, J.C.N., and Martins, E.Q.V., "A bicriterion shortest path algorithm", *European Journal of Operational Research* 11 (1982) 399–404.

[2] Corley, H.W., and Moon, I.D., "Shortest paths in networks with vector weights", *Journal of Optimization Theory and Applications* 46 (1985) 79–86.

[3] Dijkstra, E.W., "A note on two problems in connection with graphs", *Numerische Mathematik* 1 (1959) 269–271.

[4] Hansen, P., "Bicriterion path problems", in: G. Fandel and T. Gal (eds.), *Multiple Criteria Decision Making: Theory and Applications*, Lecture Notes in Economics and in Mathematical Systems 177, Springer, Heidelberg, 1980, 109–127.

[5] Henig, M.I., "The shortest path problem with two objective functions", *European Journal of Operational Research* 25 (1985) 281–291.

[6] Lawler, E.L., *Combinatorial Optimization: Networks and Matroids*, Holt, Rinehart & Winston, New York, 1976.

[7] Liu, C.L., *Elements of Discrete Mathematics*, McGraw-Hill, New York, 1977.

[8] Martins, E.Q.V., "On a multicriteria shortest path problem", *European Journal of Operational Research* 16 (1984) 236–245.

[9] Nilsson, N.J., *Problem-Solving Methods in Artificial Intelligence*, McGraw-Hill, New York, 1971.

[10] Pearl, J., *Heuristics: Intelligent Search Strategies for Computer Problem Solving*, Addison-Wesley, Reading, MA, 1984.

[11] Tung, C.T., and Chew, K.L., "A bicriterion Pareto-optimal path algorithm", *Asia-Pacific Journal of Operational Research* 5 (1988) 166–172.

[12] Vincke, Ph., "Problème multicritères", *Cahiers du Centre d'Études de Recherche Opérationelle* 16 (1974) 425–439.